

ДИСЦИПЛИНА Информационные системы управления данными и документами
(полное наименование дисциплины без сокращений)

ИНСТИТУТ информационных технологий

КАФЕДРА информационных технологий в атомной энергетике
(полное наименование кафедры)

ВИД УЧЕБНОГО
МАТЕРИАЛА Практическая работа
(в соответствии с пп 1-11)

ПРЕПОДАВАТЕЛЬ Тесла-Блозва Святослав Александрович
(фамилия, имя, отчество)

СЕМЕСТР 7 семестр 2026 – 2027 учебный год
(указать семестр обучения, учебный год)

Практическая работа №2

Тема:

Разработка формы создания документа с валидацией (Vaadin)

Цель: спроектировать экранную форму создания документа, настроить связывание данных с моделью и реализовать логику проверки вводимых значений.

Краткие теоретические сведения

Интеграция Spring Boot и Vaadin. Vaadin работает по серверной парадигме (Server-Side), запускаясь внутри единого контейнера Spring Boot. Все UI-компоненты выполняются на стороне сервера (JVM), а клиентский браузер выступает в роли «тонкого клиента», отрисовывающего интерфейс через WebSocket/HTTP. Аннотация `@Route` автоматически превращает класс представления в веб-страницу, а механизмы Spring (например, `@Service` и `@Repository`) позволяют легко внедрять бизнес-логику через конструктор (Dependency Injection).

Layouts (Компоновщики) - это контейнеры, которые определяют, как компоненты будут располагаться на экране. Основными являются `VerticalLayout` (в столбец), `HorizontalLayout` (в строку) и `FormLayout` (специальный контейнер для форм с автоматическим выравниванием подписей и полей).

Binder - важнейший механизм Vaadin, который связывает значения из полей ввода (`TextField`, `DatePicker` и др.) с полями Java-объекта (POJO). Binder берет на себя рутину по вычитке данных из интерфейса и их записи в объект.

Валидация - процесс проверки корректности введенных данных до того, как они попадут в базу. Валидаторы могут проверять как простые условия (поле не пустое), так и сложные (проверка формата регистрационного номера через регулярные выражения).

Типизированные компоненты:

`TextField` - для короткого текста.

DatePicker - для работы с датами (с учетом часовых поясов).

ComboBox - выпадающий список для выбора из заранее определенных значений (например, типов документов).

Задание:

Часть 1. Создание проекта на Start.Vaadin

Перейдите на сайт генератора проектов:

<https://start.vaadin.com/app?type=sample>

Выберите элементы, представленные на рисунке ниже. В GroupID необходимо указать com.<ваша_фамилия>. В ArtifactID указать appikbo<номер_группы>.

Скачайте архив с проектом и распакуйте его на компьютере.

Create your Vaadin app

Configure your starter below, then download it or hand the spec to your AI tool.

01 Choose your starting point

Pick how much we scaffold. You control the rest. All starters use Maven, Spring Boot, JPA and H2.

Blank canvas

Just Maven, Spring Boot, JPA and H2. Nothing else.

Sample view

A working to-do list UI that showcases Vaadin components and patterns you can learn from

Spec-driven development

Describe your app; AI drafts the use cases, data model, architecture and design system

02 Choose your edition

Vaadin

The open-source framework. Everything you need to build and ship production apps, with an optional Pro subscription for premium components & developer tools.

Vaadin Enterprise Edition

The same framework, built for business-critical apps. You get long-term maintenance, security and accessibility reviews, and support for enterprise application servers.

03 Configure the basics Optional

Sensible defaults are already filled in, so you can skip this if you're just getting started.

Vaadin Version

Vaadin 25.2 Vaadin 25.3 (pre)

Java Version

Java 21 LTS Java 25 LTS Java 27

Theme

Aura Lumo

Group ID

com.teelablozva

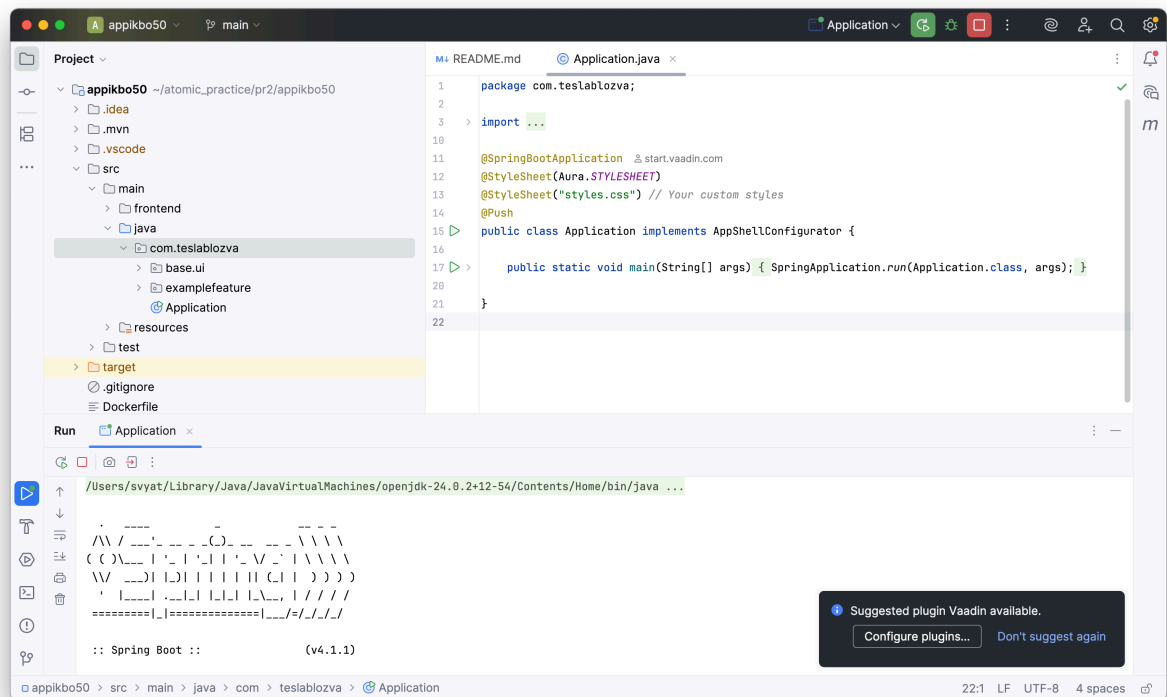
Artifact ID

appikbo504

Откройте скачанный проект в среде разработки IntelliJ IDEA. Дождитесь окончания индексации проекта и скачивания зависимостей Maven. Перейдите к главному классу приложения по пути: src/main/java/com/<ваша_фамилия>/Application.java

Нажмите кнопку запуска Run над методом main

Скриншот интерфейса IntelliJ IDEA с запущенным проектом и консолью Spring Boot представлен ниже.



После успешного запуска откройте браузер и перейдите по адресу: <http://localhost:8080/>. Должен открыться базовый интерфейс приложения Vaadin.

Часть 2. Создание класса модели Document

Создайте новый пакет documentfeature внутри вашего основного пакета (com.<ваша_фамилия>), а в нем создайте класс Document.java. Класс должен содержать следующие поля:

- Название (String);
- Регистрационный номер (String);
- Дата регистрации (LocalDate);
- Тип документа (Enum или String - «Приказ», «Распоряжение», «Служебная записка»).

Содержание класса представлено в листинге ниже:

```

package com.teslablozva.documentfeature;

import java.time.LocalDate;

public class Document {
    private String title;
    private String regNumber;
    private LocalDate regDate;
    private String type; // "Приказ", "Распоряжение", "Служебная записка"

    // Конструкторы
    public Document() {
    }

    public Document(String title, String regNumber, LocalDate regDate, String
type) {
        this.title = title;
        this.regNumber = regNumber;
        this.regDate = regDate;
        this.type = type;
    }

    public String getTitle() {
        return title;
    }

    public void setTitle(String title) {
        this.title = title;
    }

    .....
}

```

Геттеры и сеттеры напишите самостоятельно или сгенерируйте в IDE командой Alt+Insert -> Getter and Setter

Часть 3. Создание репозитория и сервиса

Для соблюдения архитектурных принципов Enterprise-разработки создадим классы-заглушки для работы с данными. Создайте интерфейс DocumentRepository.java в пакете documentfeature:

```

package com.teslablozva.documentfeature;

import org.springframework.stereotype.Repository;

import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

@Repository
public class DocumentRepository {
    // Временное хранилище в памяти (для демонстрации)
    private final List<Document> documents = new ArrayList<>();
    private long nextId = 1;

    public Document save(Document document) {
        // Простейшая имитация сохранения
        // В реальности здесь будет работа с БД
        documents.add(document);
    }
}

```

```

        return document;
    }

    public List<Document> findAll() {
        return documents;
    }

    public Optional<Document> findById(Long id) {
        // Пока не используем ID
        return Optional.empty();
    }
}

```

Создайте класс сервиса `DocumentService.java` в том же пакете:

```

package com.teslablozva.documentfeature;

import org.springframework.stereotype.Service;

@Service
public class DocumentService {
    private final DocumentRepository documentRepository;

    public DocumentService(DocumentRepository documentRepository) {
        this.documentRepository = documentRepository;
    }

    public Document saveDocument(Document document) {
        // Здесь можно добавить валидацию, логирование и т.д.
        return documentRepository.save(document);
    }
}

```

Часть 4. Создание экранной формы (View)

Создайте подпакет `ui` внутри `documentfeature`, а в нем класс

`DocumentFormView.java`:

```

package com.teslablozva.documentfeature.ui;

import com.vaadin.flow.component.button.Button;
import com.vaadin.flow.component.datepicker.DatePicker;
import com.vaadin.flow.component.formlayout.FormLayout;
import com.vaadin.flow.component.notification.Notification;
import com.vaadin.flow.component.orderedlayout.VerticalLayout;
import com.vaadin.flow.component.textfield.TextField;
import com.vaadin.flow.data.binder.Binder;
import com.vaadin.flow.data.validator.RegexpValidator;
import com.vaadin.flow.router.Route;
import com.teslablozva.base.ui.MainLayout; // если используете общий лэйаут
import com.teslablozva.documentfeature.Document;
import com.teslablozva.documentfeature.DocumentService;

import java.time.LocalDate;

@Route(value = "document-form", layout = MainLayout.class) // указываем URL и
лэйаут
public class DocumentFormView extends VerticalLayout {

```

```

private final DocumentService documentService;
private final Binder<Document> binder = new Binder<>(Document.class);

// Поля ввода
private TextField titleField = new TextField("Название");
private TextField regNumberField = new TextField("Регистрационный
номер");
private DatePicker regDatePicker = new DatePicker("Дата регистрации");
private TextField typeField = new TextField("Тип документа"); // заменить
на ComboBox

private Button saveButton = new Button("Сохранить");
private Button clearButton = new Button("Очистить");

public DocumentFormView(DocumentService documentService) {
    this.documentService = documentService;

    // Настройка формы
    FormLayout formLayout = new FormLayout();
    formLayout.add(titleField, regNumberField, regDatePicker, typeField);

    add(formLayout, saveButton, clearButton);

    // Конфигурация Binder
    binder.forField(titleField)
        .asRequired("Название не может быть пустым")
        .bind(Document::getTitle, Document::setTitle);

    binder.forField(regNumberField)
        .withValidator(new RegexValidator(
            "Номер должен соответствовать формату XXX-XX
(например, 123-45)",
            "^\\d{3}-\\d{2}$"))
        .bind(Document::getRegNumber, Document::setRegNumber);

    binder.forField(regDatePicker)
        .withValidator(date -> date != null &&
!date.isAfter(LocalDate.now()),
            "Дата регистрации не может быть позже текущей")
        .bind(Document::getRegDate, Document::setRegDate);

    binder.forField(typeField)
        .bind(Document::getType, Document::setType);

    // Обработчики кнопок
    saveButton.addClickListener(event -> saveDocument());
    clearButton.addClickListener(event -> clearForm());
}

private void saveDocument() {
    Document document = new Document();
    if (binder.writeBeanIfValid(document)) {
        // Сохраняем через сервис
        documentService.saveDocument(document);
        Notification.show("Документ успешно сохранён: " +
document.getTitle());
        clearForm(); // опционально очистить после сохранения
    } else {
        Notification.show("Пожалуйста, исправьте ошибки в форме", 3000,
Notification.Position.MIDDLE);
    }
}
}

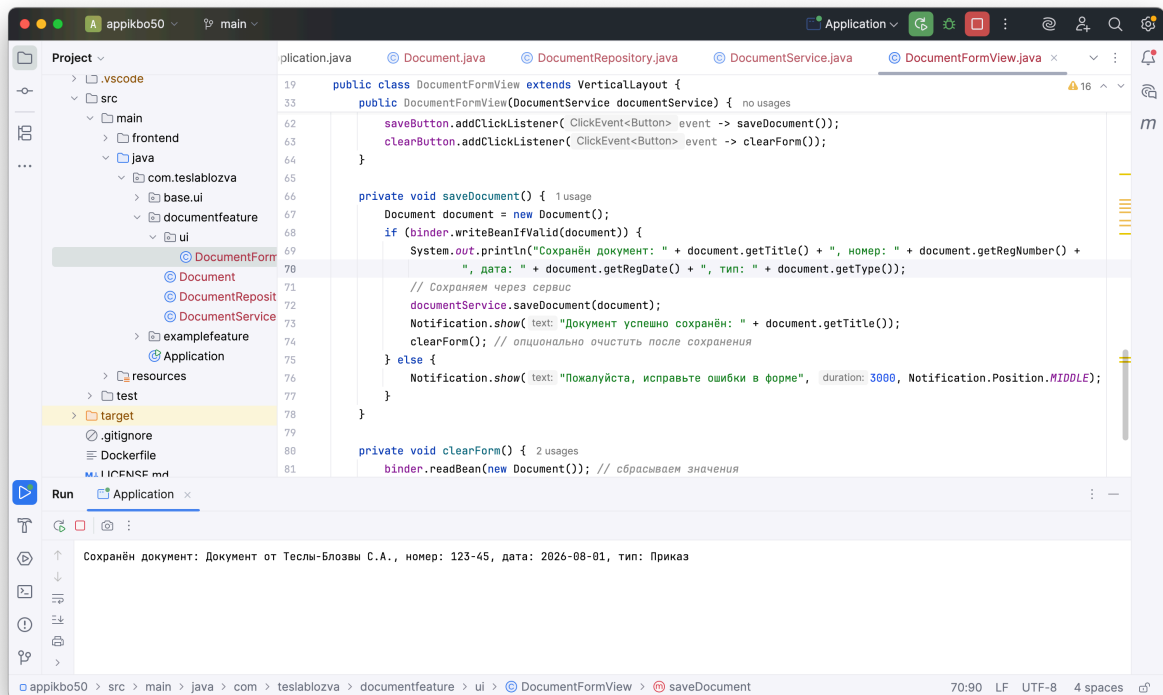
```

```
private void clearForm() {
    binder.readBean(new Document()); // сбрасываем значения
    // Можно также явно очистить поля, но readBean(new Document())
достаточно
}
}
```

Часть 5. Тестирование приложения

- Перезапустите приложение в IntelliJ IDEA. Откройте в браузере адрес: <http://localhost:8080/document-form>.
- Проверьте работу валидации, попробуйте нажать «Сохранить» с пустыми полями или ввести неверный формат номера (12-34), либо дату из будущего - система должна подсветить ошибки (сделайте скриншот).
- Введите корректные данные и нажмите «Сохранить»- появится всплывающее уведомление, а объект выведется в консоль IDE (сделайте скриншот).

Выведите содержимое заполненного объекта Document в консоль после успешной валидации, как представлено на скриншоте ниже.



Доработайте созданную форму создания документа:

1. Замените текстовое поле typeField (TextField) на выпадающий список ComboBox<String>. Задайте ему доступные значения: «Приказ», «Распоряжение», «Служебная записка».
2. Добавьте в модель Document новое поле «Срочный документ» (boolean urgent). Добавьте на форму соответствующий чекбокс (Checkbox) и настройте его связывание через Binder.
3. Добавьте вывод нового поля в логи консоли при успешном сохранении.
4. Приложите к отчету скриншоты измененного кода, скриншот формы с заполненным чекбоксом и выпадающим списком, а также скриншот терминала IDEA с информацией о сохраненном документе.

Содержание отчета

По итогам работы необходимо представить отчет, оформленный согласно СМКО МИРЭА, включающий описание выполнения каждого шага со скриншотами.

Контрольные вопросы:

1. Что такое Vaadin и в чем главное отличие его Server-Side архитектуры от классических SPA-фреймворков (Angular, React)?
2. Где физически хранится состояние дерева UI-компонентов пользовательской сессии в Vaadin?
3. Каким образом Spring Boot интегрируется с Vaadin и какую роль выполняет аннотация `@Route`?
4. Для чего предназначен класс `Binder<T>` в экосистеме Vaadin?
5. В чем разница между методами `binder.readBean()` и `binder.writeBean()`?
6. Почему в корпоративных приложениях для создания карточек и форм рекомендуется использовать `FormLayout` вместо обычного `HorizontalLayout`?
7. Каким образом можно настроить кастомную валидацию пользовательского ввода (например, проверку даты или формата строки)?
8. Как реализуется динамическое изменение доступности полей интерфейса (например, перевод полей в режим «только чтение» на основе прав доступа ABAC/ACL)?
9. Для чего применяются аннотация `@Push` и метод `ui.access()` при выполнении фоновых задач?
10. Назовите основные антипаттерны (распространенные ошибки разработчиков) при создании приложений на Vaadin.

Список полезных источников:

1. Официальная документация Vaadin, «Forms and Data Bindings» - <https://vaadin.com/docs/latest/flow/binding-data>
2. Хабрахабр, «Vaadin или как стать full-stack разработчиком»
3. Baeldung, «Introduction to Vaadin» - <https://www.baeldung.com/vaadin>